Array 28 (2025) 100528

Contents lists available at ScienceDirect

Array

journal homepage: www.elsevier.com/locate/array

Green metrics for AI: A hybrid strategy for environmental impact assessment

Salvatore Borraccia, Elio Masciari, Enea Vincenzo Napolitano *

DIETI, University of Naples Federico II, Italy

ARTICLE INFO

Keywords:
Carbon footprint estimation
Green AI
Emission tracking tools

ABSTRACT

Accurately assessing the environmental impact of Artificial Intelligence (AI) systems is essential to promoting sustainable development within the field. However, existing carbon emission tracking tools often rely on divergent assumptions, limited hardware compatibility, and inconsistent measurement granularity, leading to fragmented and often incomparable results. In this paper, we propose a hybrid methodology that integrates a weighted aggregation of ten widely-used emission estimation tools with a dynamic model-to-tool matching procedure. The weighted strategy assigns normalized importance scores to each tool based on six criteria. In parallel, the dynamic matching component identifies the most context-appropriate estimation tool for a given AI model. We evaluate our methodology on multiple architectures, including MobileNet, DenseNet, and GPT-3, deployed in different computing environments. Results demonstrate that our combined approach enhances both estimation accuracy and interpretability, outperforming single-tool assessments and offering a scalable solution for transparent carbon accounting in AI research.

1. Introduction

Artificial Intelligence (AI) is increasingly shaping core sectors of society, from healthcare to education, logistics, and climate modeling. Yet, its rapid growth raises concerns regarding its environmental footprint, particularly in light of the computational resources required to train and deploy state-of-the-art models. These concerns align with the broader agenda of *sustainable development*, as defined by the United Nations Sustainable Development Goals (SDGs), which call for balancing technological innovation with environmental responsibility.

Within this context, *Green AI* has emerged as a research movement focused on reducing the carbon emissions and energy consumption associated with AI systems. As introduced by Schwartz et al. [1], Green AI promotes not only the pursuit of higher performance, but also the measurement, reporting, and reduction of the environmental impact of AI. Recent studies have shown that training large-scale models can emit several tons of CO_2 equivalents, underscoring the urgent need for methodologies that can monitor and mitigate such impacts across different phases of the AI lifecycle.

Accurate quantification of the environmental impact of Artificial Intelligence (AI) systems is a prerequisite for the development and deployment of sustainable models. However, despite increasing attention to Green AI [2], current methodologies for estimating carbon emissions remain fragmented and lack standardization. Existing tracking tools differ significantly in terms of measurement scope, computational assumptions, and infrastructure compatibility, resulting in heterogeneous and often incomparable carbon footprint estimates [3]. Most available tools are designed around specific hardware configurations or application domains, and few offer mechanisms to dynamically adapt to varying AI workloads or execution environments. This absence of interoperability and dynamic adaptability limits the reliability of single-tool estimations, particularly as AI systems diversify across model sizes, architectures, and operational settings (e.g., cloud versus local infrastructures, single versus multi-GPU configurations). In this paper, we propose a unified framework that addresses these limitations through two complementary contributions. First, we introduce a weighted aggregation strategy that synthesizes estimates from multiple emission tracking tools, assigning weights based on six key criteria: scientific authority, precision, coverage breadth, adaptability to runtime variations, ease of integration, and computational transparency. This approach mitigates tool-specific biases and produces a more representative and stable carbon footprint estimate. Second, we develop a dynamic model-to-tool matching mechanism that selects the most appropriate emission tracker based on the specific characteristics of the AI model under analysis, including model type, hardware utilization profiles, infrastructure type, and dataset scale. This enables context-aware, model-specific estimations, enhancing both the accuracy and interpretability of environmental impact assessments. We validate our methodology through empirical evaluations involving AI models and computational setups. Our results demonstrate that the proposed hybrid approach yields carbon footprint estimates that are more consistent and robust than those

* Correspondence to: Via Claudio,21, Naples, Italy.
E-mail addresses: elio.masciari@unina.it (E. Masciari), eneavincenzo.napolitano@unina.it (E.V. Napolitano).

https://doi.org/10.1016/j.array.2025.100528
Received 12 May 2025; Received in revised form 4 August 2025; Accepted 27 September 2025
Available online 10 October 2025
2590-0056/© 2025 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

obtained through individual tracking tools, thus providing a scalable solution for comprehensive and reproducible sustainability evaluations in AI research.

2. Related work

The environmental impact of Artificial Intelligence (AI) has become a growing concern in both academic and industrial contexts. Early contributions such as Schwartz et al. distinguish between "Red AI", which prioritizes model accuracy, and "Green AI", which emphasizes computational efficiency and environmental sustainability in AI development processes [1]. An overview is provided by Wu et al. who analyze the environmental impact of AI across all development phases (data, experimentation, training, and inference) emphasizing the lifecycle emissions of hardware and the role of data centers and edge computing [4]. Dong et al. extend this discussion to a geopolitical perspective, differentiating between applied and innovative AI while advocating for region-specific sustainability strategies [5]. At the methodological level, recent surveys have focused on emerging technical solutions for sustainable AI. Notably, the review by Bolon et al. frames the concept of Green AI along three axes: AI-for-sustainability (green-by AI), energy-efficient algorithms (green-in AI), and metrics for environmental performance [6]. Similarly, the systematic review by Verdecchia et al. documents the surge of Green AI studies since 2020, highlighting the maturity and gaps in current research [2]. Kar et al. provide a PRISMA-based literature analysis, exploring AI approaches, challenges, and opportunities in sustainability contexts [7]. Specific technical investigations have addressed the components influencing emissions. Yokoyama et al. explore the role of software, hardware architecture (especially ARM-based systems), and power source carbon intensity on total energy consumption [8]. Hudaszek et al. complement this with a focus on low-impact computing infrastructure, particularly data centers and HPC systems [9]. Naidu et al. introduce privacy considerations as a trade-off, showing how differential privacy techniques can elevate computational costs and associated emissions [10]. Several studies have proposed metrics and tools to quantify carbon emissions. Bouza et al. and Garcia-Martinez et al. review available energy tracking tools and their technical assumptions, providing practitioners with guidance on estimation methods [11,12]. Open methodologies, such as the one by Delanoë et al. account not only for emissions but also for potential emission reductions due to the application of AI systems [13]. Similarly, LLMCarbon and OpenCarbonEval assess both operational and embodied carbon footprints of large language models and propose unified emission estimation frameworks [14,15]. Anselmo et al. and Salehi et al. address the emerging role of Data-Centric Green AI, emphasizing data quality and selection as levers for minimizing training overhead without sacrificing accuracy [16,17]. Omar et al. investigate ensemble learning in NLP models, linking methodological choices to observed variations in CO_2 emissions [18]. Finally, several contributions have analyzed AI's broader societal and sustainability roles. Vinuesa et al. evaluate AI's impact on the UN Sustainable Development Goals (SDGs), indicating both enabling and inhibiting effects [19], while Greif et al. employ a SWOT analysis to highlight key factors for effective and responsible AI deployment [20]. Ahmad et al. exemplify AI's potential in the renewable energy sector, showing how predictive algorithms can improve energy management from solar and geothermal sources [21]. Together, these works establish the theoretical and practical foundation for evaluating the environmental sustainability of AI systems, motivating the need for flexible, multi-faceted methodologies like the one proposed in this paper.

3. Proposed methodology

In this section, we present a methodology for estimating carbon emissions based on a weighted average of multiple tracking tools. The objective is to combine the strengths of different frameworks into a unified estimate that balances precision, adaptability, and coverage across diverse AI applications.

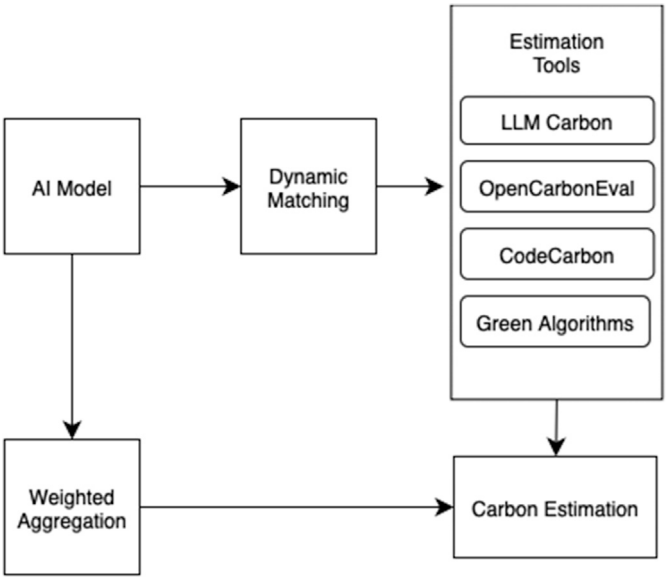


Fig. 1. Overview of the proposed methods.

To provide a clearer and more intuitive understanding of our approach, Fig. 1 illustrates the overall architecture of the proposed hybrid framework.

3.1. Weighted average framework

Each tracking tool provides a different estimate of emissions, which may vary due to differences in scope, measurement precision, or hardware assumptions. To synthesize these values, we compute a weighted average defined by the following equation:

$$E_{weighted} = \frac{\sum_{i=1}^{n} \omega_i E_i}{\sum_{i=1}^{n} \omega_i} \tag{1}$$

where E_i is the emission value of tool i and ω_i represents its assigned weight.

The weights ω_i are determined according to six criteria, selected for their relevance in assessing the reliability and usability of emission tracking tools:

- **Authority**: Degree of scientific recognition and credibility
- **Precision**: Accuracy of the emission estimates
- **Completeness**: Breadth of AI application coverage
- **Real-time adaptability**: Ability to respond dynamically to system changes
- **Ease of implementation**: Practical integration into experiments and tools
- **Accessibility**: Public availability of results and transparency of computation

The procedure consists of two phases:

(I) Determining the weights

The first phase involves determining the weights: based on the six previously described criteria, it is possible to assign weights to the respective trackers. In this process, the weights will be referred to as ω_i. Thus, ω_i represents the weight of the generic framework i, where in this case $i = 1 : n$, with $n = 10$ being the number of frameworks analyzed.

The weight assignment follows a four-step process:

1. Each framework i will be assigned a score for each of the six considered criteria, based on a scale from 0 to 1. The value will increase in constant increments of 0.10. The scores for each tracker i will be indicated as:

$$F_{i,\text{Authoritativeness}} \quad F_{i,\text{Precision}} \quad F_{i,\text{Completeness}} \quad F_{i,\text{Real-time}} \quad F_{i,\text{Implementation}} \quad F_{i,\text{Accessibility}}$$

2. For each framework i, the total score will be calculated by summing the scores obtained for each criterion. The score F_i will be given by the following formula:

$$F_i = F_{i,\text{Authoritativeness}} + F_{i,\text{Precision}} + F_{i,\text{Completeness}} + F_{i,\text{Real-time}} + F_{i,\text{Implementation}} + F_{i,\text{Accessibility}}, \quad i = 1 : n, \quad n = 10. \tag{2}$$

3. A normalization process will then be applied to the scores. The total score F_i for each framework i will be normalized by dividing it by the maximum total score obtained among all frameworks. The normalized score ω_i will be given by the following formula:

$$\omega_i = \frac{F_i}{\sum_{j=1,\ldots,n} F_j}, \quad i = 1 : n, \quad n = 10. \tag{3}$$

(II) Calculation of the weighted average of carbon emissions

Once the weights are assigned, the weighted average of carbon emissions is computed. This involves collecting the emission values E_i (expressed in $kgCO_2e$) from each tracking framework and applying Eq. (1). This step enables a more stable and representative emission estimate by mitigating biases associated with individual tools.

3.1.1. Application of the method: an example of the weight assignment procedure

An example of possible value assignments is given relative to each contribution defined in this section. The latter in fact may also vary depending on the context, the geographical region in which the model is being applied, the type of hardware applied, the presence or absence of cloud instances and many other factors.

Based on certain characteristics, such as the ability to simultaneously consider the energy consumption of multiple hardware devices, compatibility with measurements from instruments like wattmeters [11], authoritativeness and citation in scientific publications, and extension to the cloud domain, a score is assigned to each framework for each criterion.

Rationale for the selection of estimation tools

The selection of the ten emission estimation tools included in our weighted aggregation strategy was guided by a set of well-defined criteria to ensure both comprehensiveness and representativeness. We prioritized tools that are:

- **Recognized in the scientific community**, with published documentation or peer-reviewed references;
- **Technically diverse**, covering different scopes such as hardware-specific measurement (e.g., CPU, GPU, RAM), runtime energy estimation, and lifecycle analysis;
- **Compatible with both cloud-based and on-premise infrastructures**, enabling broader applicability across AI deployment contexts;
- **Capable of addressing different AI domains**, including machine learning, deep learning, natural language processing (NLP), and large language models (LLMs);
- **Complementary in methodology**, including both direct measurement (e.g., via RAPL or wattmeter-based inference) and model-based estimation using emission coefficients.

These tools have also been discussed and reviewed in recent surveys (e.g., [11]; [12]), and their relevance has been confirmed by their frequent use in AI sustainability analyses. The scoring framework that follows (see Table 1) quantitatively reflects this selection logic by evaluating each tool across six dimensions: *authority*, *precision*, *completeness*, *real-time adaptability*, *ease of implementation*, and *accessibility*.

Moreover, a cross-mapping of tools and their suitability for specific AI domains and execution environments is reported in Table 4, further demonstrating the complementarity of the selected frameworks and the need for a hybrid evaluation approach.

At this point, it is possible to proceed to the next steps and sum the total scores for each framework, obtaining the following formula:

$$\sum_{j=1}^{n} 4.9 + 4.4 + 3.3 + 4.5 + 3.1 + 3.4 + 5.1 + 1.5 + 1.4 + 2.8 = 34.4 \tag{4}$$

Now, we can proceed to calculate the weights of each framework, applying the following formula:

$$\omega_i = \frac{F_i}{\sum_{j=1}^{n} F_j} \tag{5}$$

The weights determined are shown in the table below

Once the weights have been calculated empirically, it is necessary to proceed to the definition of the weighted average. To do so, the procedure must include the quantification of the emission for each tracker considered.

A specific example is referenced in which only one GPU is used for simplicity. The considered data are as follows: (the calculation can be extended to the case of multiple GPUs; however, some tools such as MLCO2 or Cumulator can perform measurements only in the case of a single GPU, so these data are considered for computational consistency):

- **Hardware type**: NVIDIA GTX TITAN X GPU (with a TDP of 250 W, according to NVIDIA's manufacturer specifications)
- **Usage time (Runtime)**: 100 h
- **CPU**: Intel Core i9-9900 (TDP = 65 W, according to Intel specifications)
- **RAM**: 32 GB

The following utilization factors, entirely hypothetical, are considered: 80% GPU, 50% CPU, 40% RAM, unless specific tracker conditions apply. Some trackers, for example, assume full GPU utilization, while others do not consider the utilization factor at all.

For frameworks that also operate in cloud infrastructure, the following common data can be selected:

- **Provider**: Amazon Web Services (AWS)
- **Computation region (data center location)**: Europe - France (Paris) or `europe-west1`
- **Emission coefficient CI for the considered region**:

$$CI_{\text{average, France (Paris), 2023}} = 56 \ gCO_2e/kWh \tag{6}$$

a variable value that can also be consulted in *Electricity Map*.

CodeCarbon The energy consumption of the CPU, GPU, and RAM is taken into consideration. As previously stated, the Intel Core i9-9900 CPU is considered, which is also listed in the CodeCarbon GitHub repository, with a TDP of 65 W and an estimated average utilization factor of 50%, as also assumed by CodeCarbon [11]. For the GPU, the general data previously selected is used, with a NVIDIA GTX TITAN X architecture. As for the RAM, an energy consumption of 0.375 W/GB is considered . A 32 GB RAM is examined, with a utilization rate of 40%. Considering that CodeCarbon also measures cloud instances, an average emission intensity value for AWS in the France region is assumed to be 56 g CO2e/kWh. This value is supported by the GitHub library of CarbonTracker, a general value that can be applied as an average reference for any provider. This emission coefficient

Table 1
Allocation of scores for each criterion.

Framework	Authority	Precision	Complet.	Real-time	Impl.	Access.	Fi
CodeCarbon [22]	0.90	0.90	0.80	0.70	0.80	0.80	4.9
MLCO2 [23]	0.70	0.80	0.20	0.90	0.90	0.90	4.4
Experiment Impact Tracker [24]	0.70	0.70	0.80	0.20	0.40	0.50	3.3
CarbonTracker [25]	1.00	0.70	0.70	0.60	0.80	0.70	4.5
Cumulator [26]	0.70	0.70	0.20	0.50	0.50	0.50	3.1
Eco2AI [27]	0.70	0.70	0.50	0.50	0.50	0.50	3.4
Green Algorithms [28]	0.80	0.80	1.00	0.80	0.90	0.80	5.1
LLMCarbon [14]	0.20	0.70	0.10	0.10	0.20	0.20	1.5
OpenCarbonEval [15]	0.10	0.70	0.10	0.10	0.20	0.20	1.4
Cloud Instances [29]	0.70	0.70	0.40	0.20	0.40	0.40	2.8

Table 2
Table with frameworks and respective weights.

Framework	Weight (ω)
CodeCarbon	0.142
MLCO2	0.128
Experiment Impact Tracker	0.096
CarbonTracker	0.131
Cumulator	0.090
Eco2AI	0.099
Green Algorithms	0.148
LLMCarbon	0.044
OpenCarbonEval	0.041
Istances Cloud	0.081

incorporates the respective energy mixes defined in the document [22], which are compatible with the values present in the GitHub library of CodeCarbon. In the case of CodeCarbon, the GPU utilization factor is not considered as a hypothesis; therefore, the utilization factor will only be reported for the CPU and memory. We arrive at the determination of the following values regarding the contribution of GPU, CPU, and RAM through the following formulas:

$$E_{GPU} = 250\ \text{W} \times 100\ \text{h} = 25000\ \text{Wh} = 25\ \text{kWh} \tag{7}$$

$$E_{CPU} = 65\ \text{W} \times 0.5 \times 100\ \text{h} = 3250\ \text{Wh} = 3.25\ \text{kWh} \tag{8}$$

$$E_{RAM} = 0.375\frac{\text{W}}{\text{GB}} \times 32\ \text{GB} \times 0.4 \times 100\ \text{h} = 480\ \text{Wh} = 0.48\ \text{kWh} \tag{9}$$

At this point, it is possible to calculate the CO_2e emissions value for CodeCarbon. It is important to highlight that CodeCarbon usually does not take into account the PUE (Power Usage Effectiveness); however, for analyzing cloud providers, such as AWS in this case, an exception is made, considering it. In this analysis, a standard PUE value of **1.58** is used. Thus, the contribution of the emission intensity for AWS and for the considered region is exploited, obtaining the following result by applying Eq. (10) of the document:

$$E_1 = 56\frac{\text{gCO}_2\text{e}}{\text{kWh}} \times 1.58 \times (25\ \text{kWh} + 3.25\ \text{kWh} + 0.48\ \text{kWh}) \tag{10}$$

$$E_1 = 2542\ \text{gCO}_2\text{e} = 2.54\ \text{kgCO}_2\text{e} \tag{11}$$

MLCO2 For MLCO2 there is a special online calculator,[1] which by entering the data considered allows the instantaneous calculation of emissions. It can be viewed with the input data input interface expressed at the beginning of the section. In particular, it returns a result equal to:

$$E_2 = 2.5\,\text{kg}\ \text{CO}_2\text{e} \tag{12}$$

It is important to consider the fact that MLCO2 by assumption does not measure RAM or even the CPU [11] and therefore will not be present in the online.

[1] https://www.mlco2.com.

Experiment-Impact-Tracker Since the CPU is of the Intel type and has access to RAPL files, and supports the Linux operating system, it is possible to consider its energy consumption according to EIT, also utilizing the data available in the EIT's dedicated GitHub library. Therefore, the TDP_{CPU} is set to 65 W. However, for the memory, EIT uses the DRAM in its formula. For the DRAM, it is assumed the use of 2 modules of DDR4 memory from Crucial. In fact, Crucial specifies that a typical power consumption for the memory is about 5 W per module. For the GPU, the value remains the same: $\text{TDP}_{GPU} = 250$ W. The respective usage percentages for each resource are hypothesized, considering they can operate independently of each other and under the same general assumptions. The usage factors are as follows:

$$p_{gpu} = 80\% = 0.8 \tag{13}$$

$$p_{cpu} = 50\% = 0.5 \tag{14}$$

$$p_{dram} = 40\% = 0.4 \tag{15}$$

as also assumed for CodeCarbon.

Subsequently, to calculate the energy usage of each resource, their respective TDPs are utilized:

$$e_{gpu} = TDP_{gpu} \times 100\ \text{h} = 250\ \text{W} \times 100\ \text{h} = 25000\ \text{Wh} = 25\ \text{kWh} \tag{16}$$

$$e_{cpu} = TDP_{cpu} \times 100\ \text{h} = 65\ \text{W} \times 100\ \text{h} = 6500\ \text{Wh} = 6.5\ \text{kWh} \tag{17}$$

$$e_{dram} = TDP_{dram} \times 100\ \text{h} = 2 \times 5\ \text{W} \times 100\ \text{h} = 1000\ \text{Wh} = 1\ \text{kWh} \tag{18}$$

At this point, it is possible to proceed by applying formula 3.3 of the paper, obtaining the following result, adopting the standard PUE equal to PUE = 1.58, and inserting the calculated values for the individual hardware systems:

$$e_{total} = 1.58 \times (0.8 \times 25\ \text{kWh} + 0.5 \times 6.5\ \text{kWh} + 0.4 \times 1\ \text{kWh}) = 37.37\ \text{kWh} \tag{19}$$

At this point, we proceed to calculate the Carbon footprint, using $CI_{media,France,2023} = 56\ \text{gCO}_2\text{e/kWh}$ relative to the France region present in the CarbonTracker library, resulting in the following result:

$$E_3 = 37.37\ \text{kWh} \times 56\frac{\text{gCO}_2\text{e}}{\text{kWh}} = 2092.72\ \text{gCO}_2\text{e} = 2.09\ \text{kgCO}_2\text{e} \tag{20}$$

And the overall emissions value was obtained through the EIT tracker.

According to the tracker, with the available data, it is already possible to calculate the energy E by applying formula 3.8 to all three components: GPU, CPU, and RAM, resulting in the following formulas:

$$E_{GPU} = 1.58 \times 250\ \text{W} \times 100\ \text{h} = 39500\ \text{Wh} = 39.5\ \text{kWh} \tag{21}$$

Carbon Tracker It is important to emphasize that Carbon Tracker does not consider the utilization factor for the GPU, as it considers the consumption for the entire machine [11]. The same applies to the CPU, and therefore the utilization factor is not considered. Being an Intel CPU with access to the RAPL file, the framework also tracks the contribution related to the CPU and subsequently to the 32 GB RAM, again without the utilization factor.

$$E_{CPU} = 1.58 \times 65\ \text{W} \times 100\ \text{h} = 10270\ \text{Wh} = 10.27\ \text{kWh} \tag{22}$$

Assuming the use of two 5 W DRAM DDR4 modules, we have:

$$E_{RAM} = 1.58 \times 2 \times 5 \text{ W} \times 100 \text{ h} = 1580 \text{ Wh} = 1.58 \text{ kWh} \quad (23)$$

At this point, the average carbon intensity for the France and Central Europe region for 2023 is considered, denoted as $CI_{\text{media, France, 2023}} = 56 \text{ gCO}_2\text{e/kWh}$ (data from the Carbon Tracker GitHub library for 2023, a general value but applicable as an average data point for any provider). Denoting the Carbon Footprint as E_4, we arrive at the determination of the carbon footprint through the Carbon Tracker expressed by the formula:

$$E_4 = CI_{\text{media, France, 2023}} \times E$$
$$= (39.5 \text{ kWh} + 10.27 \text{ kWh} + 1.58 \text{ kWh}) \times 56 \text{ gCO}_2\text{e/kWh} \quad (24)$$

$$E_4 = 2875.6 \text{ gCO}_2\text{e} = 2.88 \text{ kgCO}_2\text{e} \quad (25)$$

This represents the CO_2e value for the Carbon Tracker tool.

Cumulator For this calculation, both the computational cost and the communication cost were considered. For the computational cost, data related to the GTX TITAN X with a TDP of 250 W, a time of 100 h, and the same value of $C_{\text{media, Francia, 2023}} = 56 \text{ gCO}_2\text{e/kWh}$ used in CarbonTracker were taken into account. It should be emphasized that Cumulator does not measure memory consumption and is also unable to simultaneously measure the energy consumption of both the CPU and GPU. Therefore, in this discussion, only the energy consumption of the GPU as measured by Cumulator will be considered.

For the communication cost, the 1-byte relative model will be used, with an estimated value of 6.894×10^{-11} kWh/byte [26,30]. Thus, the calculation proceeds with two independent costs. The GPU utilization factor is not considered since it is assumed that the GPU is operating under maximum load [11].

- **Computation cost:** It is indicated as CP and results as follows:

$$CP = 100 \text{ h} \times 0.250 \text{ kW} \times \frac{56 \text{ gCO}_2\text{e}}{\text{kWh}} = 1400 \text{ gCO}_2\text{e} = 1.4 \text{ kgCO}_2\text{e} \quad (26)$$

- **Communication cost:** It is indicated as CC and results as follows:

$$CC = 1 \text{ byte} \times 6.894 \times 10^{-11} \frac{\text{kWh}}{\text{byte}} \times \frac{56 \text{ gCO}_2\text{e}}{\text{kWh}} = 3.86 \times 10^{-9} \text{ gCO}_2\text{e} \quad (27)$$

$$= 3.86 \times 10^{-12} \text{ kgCO}_2\text{e} \quad (28)$$

The total emission value is given by the sum of these two contributions; however, in this case, the communication cost is negligible, so the total emissions calculated with Cumulator coincide with the computational cost:

$$E_5 \approx CP = 1.4 \text{ kgCO}_2\text{e} \quad (29)$$

Eco2AI The consumption of GPU, CPU, and RAM is considered. The integral method is used to account for power variations over a given time interval. In this case, the assumption of constant TDP will be made. This tool is used with a predefined PUE value of 1. The relevant calculations are presented below:

- **GPU:** The value $\text{TDP}_{GPU} = 250$ W is considered, with $t = 100$ h, and by applying equation 3.13 in the case of constant load, we obtain:

$$E_{GPU} = 250 \text{ W} \times 100 \text{ h} = 25000 \text{ Wh} = 25 \text{ kWh} \quad (30)$$

In this case, the utilization factor is not considered because Eco2AI accounts for the entire machine's consumption [11].

- **CPU:** The same CPU defined in the given data is used, as specified in the Eco2AI library on GitHub, namely the Intel Core i9-9900 with a TDP of $\text{TDP}_{CPU} = 65$ W. Since W_{CPU} is the average fractioned load, assuming a 50% load, we set $W_{CPU} = 0.5$. We obtain:

$$E_{CPU} = 65 \text{ W} \times 0.5 \times 100 \text{ h} = 3250 \text{ Wh} = 3.25 \text{ kWh} \quad (31)$$

- **RAM:** In this case, equation is considered with the assumption of a constant load over time, with a value of 0.375 W/Gb representing the average energy consumption of DDR3 and DDR4 memory [27]. For allocated memory M_{RAM}, an assumption of 32 Gb is made, with an estimated utilization rate of 40

$$E_{RAM} = 0.375 \frac{\text{W}}{\text{Gb}} \times 32 \text{ Gb} \times 0.4 \times 100 \text{ h} = 480 \text{ Wh} = 0.48 \text{ kWh} \quad (32)$$

At this point, the emission intensity coefficient γ (kgCO_2/MWh) is defined. Specific values are used as defined and related references; in this case, the relative value for France is considered, given by

$$\gamma = 67.53 \frac{\text{kgCO}_2\text{e}}{\text{MWh}} \quad (33)$$

The CO_2 emissions value is obtained as:

$$E_6 = 67.53 \frac{\text{kgCO}_2\text{e}}{\text{MWh}} \times \frac{1}{1000} \times (25 \text{ kWh} + 3.25 \text{ kWh} + 0.48 \text{ kWh}) \quad (34)$$

$$= \frac{\text{kgCO}_2\text{e}}{\text{kWh}} \times (25 \text{ kWh} + 3.25 \text{ kWh} + 0.48 \text{ kWh}) \quad (35)$$

$$= 1.94 \text{ kgCO}_2\text{e} \quad (36)$$

Green Algorithms It is possible to use the online calculator in this case to account for emissions. The online calculator, https://calculator.green-algorithms.org/, allows the calculation of consumption by considering both GPU and CPU. The pragmatic scaling factor is not considered, as its determination would be too complex, depending on many factors and varying from one algorithm to another. The calculator provides the option to account for both GPU and CPU consumption and to include the relevant service provider along with its corresponding location. By entering the GPU and CPU types of the hypothesis along with their respective TDP values, the runtime (100 h), and all relevant information concerning the hypothetical cloud infrastructure into the appropriate calculator, the software returns a result of 2.01 kgCO2e. This value of 2.01 kgCO2e is related to the considered GPU and CPU. This value of 2.01 kgCO2e is related to the considered GPU and CPU. However, the energy consumption of the RAM must also be considered. The energy consumption of the memory is 0.3725 W/GB; for the hypothesis, 32 GB of RAM is considered, assuming a 40% usage factor and a CImedia, France(Paris), 2023 = 56 gCO2e/kWh:

$$E_{\text{RAM}} = \frac{0.3725 \text{ W}}{\text{GB}} \times \frac{56 \text{ gCO2e}}{\text{kWh}} \times 0.4 = 26.7 \text{ gCO2e}$$
$$= 0.027 \text{ kgCO2e} \times 32 \text{ GB} \times 100 \text{ ore} \times \frac{56 \text{ gCO2e}}{\text{kWh}} \times 0.4 = 0.3725 \text{ kgCO2e} \quad (37)$$

Therefore, the total emission value for Green Algorithms will be:

$$E_7 = 2.01 \text{ kgCO2e} + 0.027 \text{ kgCO2e} = 2.04 \text{ kgCO2e} \quad (38)$$

LLMCarbon Data from the LLMCarbon GitHub library is utilized. The process begins with the determination of the overall energy of the hardware units; the use of CPU, GPU, and RAM is considered to execute and train a large language model, specifically GPT-3. The hardware efficiency defined for this system is eff = 0.197 [14]. Consequently, the following formula can be applied:

$$energy_{oper} = 4.67 \text{ kWh} \cdot 1.1 = 5.14 \text{ kWh} \quad (39)$$

Considering the carbon intensity of reference for the Western Europe region, particularly France, the formula to calculate the operational CO_2e emissions, denoted as CO, can be applied:

$$\text{CO} = 5.14 \text{ kWh} \cdot \frac{56 \text{ gCO}_2\text{e}}{\text{kWh}} = 287.84 \text{ gCO}_2\text{e} = 0.29 \text{ kgCO}_2\text{e} \quad (40)$$

Subsequently, the calculation of embodied carbon proceeds, considering parameters such as chip area and carbon emitted per unit area, defined as CPA; the relative values are present in the table in the LLMCarbon GitHub library for GPU, CPU, and DRAM. The table provides the area and CPA values, but the CO_2e value related to the chip for all three hardware devices is directly provided.

$$CO_{2e_{chip,GPU}} = 148.2\ \text{kgCO}_2\text{e} \tag{41}$$

$$CO_{2e_{chip,CPU}} = 1.47\ \text{kgCO}_2\text{e} \tag{42}$$

$$CO_{2e_{chip,DRAM}} = 102.4\ \text{kgCO}_2\text{e} \tag{43}$$

The calculation of the embodied carbon footprint CE considering a lifetime value for the hardware of 1 year for all (corresponding to 8760 h) for simplicity is expressed by formula:

$$\text{CE} = \frac{100\ \text{h} \cdot 148.2\ \text{kgCO}_2\text{e} \cdot 0.8}{8760\ \text{h}} + \frac{100\ \text{h} \cdot 1.47\ \text{kgCO}_2\text{e} \cdot 0.5}{8760\ \text{h}} + \frac{100\ \text{h} \cdot 102.4\ \text{kgCO}_2\text{e} \cdot 0.4}{8760\ \text{h}} = 1.83\ \text{kgCO}_2\text{e} \tag{44}$$

The total carbon is obtained as:

$$E_8 = \text{CE} + \text{CO} = 1.83\ \text{kgCO}_2\text{e} + 0.29\ \text{kgCO}_2\text{e} = 2.12\ \text{kgCO}_2\text{e} \tag{45}$$

OpenCarbonEval A distinction is made between operational carbon footprint and embedded or embodied carbon footprint, and then the two contributions are summed.

CO indicates the incorporated carbon footprint, and CE indicates the embedded carbon footprint. Applying formula with the initial data yields the following formula:

$$\text{CO} = \left(\frac{250\ \text{kW}}{1000} \cdot 0.8\right) \cdot 100 \cdot \frac{56\ \text{kgCO}_2\text{e}}{1000\ \text{kWh}} = 1.12\ \text{kgCO}_2\text{e} \tag{46}$$

Regarding lifetime, a 1-year lifespan is typically considered for each hardware component; one year is considered to be 8760 h. Considering, for example, an applied StartCoder model, with this GPU load, for analyses with an embodied carbon value Cr = 480.38 kgCO_2e [15], the formula for CE can be applied to obtain formula 4.56:

$$CE = \frac{100\ \text{h}}{8760\ \text{h}} \cdot 0.8 \cdot (480.38\ \text{kgCO}_2\text{e}) = 4.39\ \text{kgCO}_2\text{e} \tag{47}$$

The sum represents the total emissions:

$$E_9 = 4.39\ \text{kgCO}_2\text{e} + 1.12\ \text{kgCO}_2\text{e} = 5.51\ \text{kgCO}_2\text{e} \tag{48}$$

Only the calculations relating to the GPU are considered, as per the theoretical formula present in OpenCarbonEval, since it is based on the GPU's energy consumption and the use of GPUs in parallel to support models like StarterCoder.

Cloud Instances In this case, the cloud infrastructure under consideration is represented by Amazon Web Services, with data centers located in the France region. The infrastructure consists of a GTX TITAN X GPU and an Intel i9-9900 CPU. The calculations are performed as in previous cases, considering the estimated utilization factors. The emission calculation principles follow previous examples [29]. Two DDR4 RAM modules, each consuming 5 W, are also included, leading to the following formulas:

$$E_{\text{GPU}} = 100\ \text{h} \times 250\ \text{W} \times 0.8 = 20000\ \text{Wh} = 20\ \text{kWh} \tag{49}$$

$$E_{\text{CPU}} = 100\ \text{h} \times 65\ \text{W} \times 0.5 = 3250\ \text{Wh} = 3.25\ \text{kWh} \tag{50}$$

$$E_{\text{RAM}} = 2 \times 5\ \text{W} \times 0.4 \times 100\ \text{h} = 400\ \text{Wh} = 0.40\ \text{kWh} \tag{51}$$

The total energy is then considered, assuming a standard PUE of 1.58:

$$E_T = (20\ \text{kWh} + 3.25\ \text{kWh} + 0.40\ \text{kWh}) \times 1.58 = 37.37\ \text{kWh} \tag{52}$$

Table 3
Comparison of emission estimation trackers.

GTX TITAN, Intel Core i9-9900, RAM 32 GB, 100 h, Europe (Paris)		
Tracker	Weights (ω_i)	Emissions (kgCO_2e)
CodeCarbon	0.142	2.54
MLCO_2	0.128	2.50
Experiment Impact Tracker	0.096	2.09
CarbonTracker	0.131	2.88
Cumulator	0.090	1.40
Eco2AI	0.099	1.94
Green Algorithms	0.148	2.04
LLMCarbon	0.044	2.12
OpenCarbonEval	0.041	5.51
Cloud Instances	0.081	2.10

At this point, the emission coefficient for cloud infrastructure is applied to determine the total emissions:

$$E_{10} = 37.37\ \text{kWh} \times 56 \frac{\text{gCO}_2\text{e}}{\text{kWh}} = 2092.72\ \text{gCO}_2\text{e} = 2.10\ \text{kgCO}_2\text{e} \tag{53}$$

The results of the application calculation are summarized in the following Table 3

Calculation of the Weighted Average

At this point, it is possible to proceed with the calculation of the weighted average. Since the sum of the weights is already equal to one, and therefore already normalized, it is possible to apply the following formula:

$$E_{\text{weighted}} = \frac{\sum_{i=1}^{n} \omega_i E_i}{\sum_{i=1}^{n} \omega_i} = (0.142 \times 2.54) + (0.128 \times 2.50) + (0.096 \times 2.09) + (0.131 \times 2.88) + (0.090 \times 1.40) + (0.099 \times 1.94) + (0.148 \times 2.04) + (0.044 \times 2.21) + (0.041 \times 5.51) + (0.081 \times 2.10) = 2.37\ \text{kgCO}_2\text{e} \tag{54}$$

Justification of evaluation criteria

The six criteria used for scoring each emission tracking tool were selected to capture a comprehensive and multidimensional evaluation of their applicability and robustness. Below, we explain the rationale behind each dimension:

- **Authority**: Reflects the scientific credibility of the tool, based on peer-reviewed publications, adoption in other studies, and citation count.
- **Precision**: Assesses the degree to which the tool provides accurate and reproducible emission estimates, particularly regarding granularity and hardware-specific measurements.
- **Completeness**: Measures the breadth of components considered (e.g., GPU, CPU, RAM, cloud usage, embodied emissions) and the ability to capture the full computational footprint.
- **Real-time adaptability**: Indicates the tool's capability to track and update emissions dynamically during runtime, adapting to workload changes.
- **Ease of implementation**: Considers practical factors such as integration effort, API usability, and compatibility with standard ML workflows.
- **Accessibility**: Evaluates the availability of the tool (open-source or not), transparency of formulas and assumptions, and ease of result interpretation.

Each tool was scored on a normalized scale from 0 to 1 (in increments of 0.10) for each criterion. The assignment was based on a systematic review of publicly available documentation, source code analysis, empirical studies, and survey literature (e.g., [11,12]).

We recognize that some subjectivity is unavoidable in such assessments. However, our method follows a structured and reproducible approach. All assigned scores and corresponding justifications are reported in Table 2, enabling transparency and repeatability. Future work may include data-driven or community-informed scoring schemes to further enhance the objectivity of the weighting system.

3.2. Dynamic model matching

The objective is to develop a procedure capable of identifying a specific algorithm or model operating within a particular field of artificial intelligence and directly assigning it to the corresponding framework. Among the ten frameworks considered in this study, each is specifically designed to evaluate a particular type of task and accurately measure its environmental impact in terms of emissions.

To ensure the framework's adaptability and dynamic characteristics, it is first necessary to define examples of algorithms functioning within the main areas of artificial intelligence. In this study, the most relevant areas have been identified as follows:

1. Machine Learning (ML)
2. Deep Learning (DL)
3. Natural Language Processing (NLP)
4. Large Language Models (LLMs)

This methodology assigns AI models to the most suitable emission estimation framework based on:

- AI model type (ML, DL, NLP, LLMs)
- Computational infrastructure (local vs. cloud-based execution)
- Dataset size and hardware utilization
- Support for multi-GPU environments

The new procedure consists of two main phases:

1. **Phase 1**: Since it is necessary to determine which tool to apply based on the given context, it becomes essential to establish principles that relate the elements of analysis.
2. **Phase 2**: To better understand the methodology, a practical example can be conducted, where the principles established in the first phase are applied.

The criteria for defining these rules are based on the following elements:

- First, it is necessary to determine the area of AI in which the algorithm is implemented.
- The way the algorithm is implemented: whether it runs on multiple GPUs, through measurements with cloud-based structures, or exclusively with GPU hardware infrastructures. Additionally, it is important to specify whether a CPU processor and/or RAM memory is also utilized. Furthermore, it is crucial to indicate the type of dataset on which the algorithm has been trained.

Based on these elements, an analysis will be conducted to determine which of the ten emission measurement frameworks is the most suitable. In particular, the following aspects will be considered:

- Which areas of AI each framework supports (whether it covers multiple types or is specific to a given type).
- Whether a given tool supports cloud infrastructure or relies exclusively on local hardware infrastructures (GPU, CPU, RAM memory).
- Whether a specific tracker allows measurement in the presence of multiple local infrastructures, meaning it enables energy consumption measurement when multiple GPUs operate in parallel.
- Whether a given tool accounts for energy consumption not only from the GPU but also from the CPU and RAM memory.
- Whether the analyzed measurement tool can simultaneously consider both GPU and CPU or if it imposes a single-measurement constraint for each processor.
- Whether the framework allows GPU energy consumption measurement only for specific manufacturers (e.g., NVIDIA) or particular CPU models (such as Intel processors).
- Whether the tool is compatible with the type of dataset on which the algorithm is implemented.

Based on the characteristics of the reference frameworks in the literature and the defined analytical elements, the following associations have been established between the group of reference trackers and the group of algorithms/models operating in the four main areas of artificial intelligence, as summarized in the following Table 4

In Table 4 mentioned above, the cell filled with *Specific* indicates that the tracker was specifically designed for application to that particular type of architecture. In contrast, *Adaptable* signifies that, while the tool was not originally developed for that specific condition, it possesses sufficient versatility to expand its library and adapt to measurements in other areas that are similar but exhibit distinct characteristics.

Example of Method Application

Once the main characteristics of the pairings have been listed, a practical application of this approach can be defined. An example of an algorithm or model operating in one of these fields is considered, followed by an analysis of the most suitable frameworks for assessing its environmental impact based on the characteristics outlined in the text. Three practical examples will be presented, incorporating the elements described thus far.

Consider the analysis of the **BERT-base model (Transformer)** within an NLP architecture, which exhibits the following characteristics:

- It consists of approximately 110 million parameters.
- It is assumed to be trained on a specific dataset, the "IMDB dataset", which is primarily used for processing movie review texts.
- The training time is estimated to be 12 h.
- A cloud-based infrastructure is utilized, specifically Google Cloud Platform, located in *Europe west-1*, with a renewable energy mix of approximately 30
- To support an algorithm with such a high number of parameters, an NVIDIA A100 GPU with around 80 GB of memory and a TDP ranging between 300–400 W is selected as the hardware infrastructure.

This example serves to illustrate how the matching engine selects tools based on workload metadata and environmental constraints, rather than to benchmark the precision of individual estimators. The final value should therefore be interpreted as the output of a context-sensitive fallback mechanism, not as a claim of superior accuracy. A full numerical comparison is beyond the scope of this example and is left for future work.

Following the association rules, the following analyses can be conducted: Given that the model has considerable size and a large number of parameters, it is preferable to select a framework that is more adaptable to these requirements. Since this is an NLP model implemented on a cloud infrastructure the remaining alternatives are *Green Algorithms* and *Cloud Instances*. In particular, *Green Algorithms* is highly suited for an NVIDIA A100 GPU, as it is included in the list of supported graphical units within the framework's online calculator. Additionally, it supports Google Cloud Platform by accounting for both the energy consumption of the cloud provider and the data center location, incorporating various emission coefficients into a dedicated library. *Green Algorithms* is also highly versatile when applied to NLP tasks and has already been tested with the BERT-base algorithm in multiple fields, including text mining in the biomedical sector and scientific testing [28]. Moreover,

Table 4
Matching of AI models with tracking frameworks.

Framework	ML	DL	NLP	LLM	Cloud	Multiple GPUs
CodeCarbon	Specific	Yes	Adaptable	Adaptable	Yes	Adaptable
MLCO2	Specific	Yes	Yes	No	Yes	No
EIT	Specific	Yes	Adaptable	Adaptable	No	Yes
CarbonTracker	Yes	Specific	Adaptable	Adaptable	No	No
Cumulator	Specific	Yes	No	No	No	No
Eco2AI	Yes	Specific	Yes	Specific	No	Yes
Green Algorithms	Yes	Yes	Yes	No	Yes	Yes
LLMCarbon	No	No	Yes	Specific	Yes	Yes
OpenCarbonEval	No	No	Adaptable	Specific	Yes	No
Istanze Cloud	Specific	Yes	Specific	No	Specific	No

it demonstrates high efficiency for training on datasets such as IMDB, which contain a substantial amount of information.

As an alternative, based on the selected rules, the *Cloud Instances* framework [29] can be considered for measuring this model. In their study, the author also performs training on the same BERT-base algorithm using an NVIDIA GPU.

Consequently, the methodology has identified the following trackers:

- *Green Algorithms*
- *Cloud Instances*

At this point, if the primary focus is on the energy consumption of the hardware infrastructure, particularly the GPU, in a more detailed and comprehensive manner — especially considering a large dataset — then *Green Algorithms* is the preferred choice. Conversely, if the focus is more on cloud-related factors, specifically the energy consumption associated with the provider and geographic location, the *Cloud Instances* method is more suitable.

Implementation logic of the dynamic matching procedure

The dynamic model-to-tool matching procedure is based on a rule-based system that evaluates each AI workload against a set of compatibility rules with the available estimation tools. The matching is driven by the following inputs:

- **Model class and workload type** (e.g., LLM, vision model, tree-based classifier);
- **Execution environment** (on-premise vs. cloud, energy trace availability);
- **Tool scope and prerequisites** (e.g., requirement for hardware-level access, cloud instance metadata).

We define a set of deterministic rules that guide the selection. For example:

- If a model is run on the cloud and the platform provides energy metrics, tools like `CloudEstimator` and `OpenCarbonEval` are prioritized.
- If the model is a large transformer and tool-specific support is available, `LLMCarbon` is activated.
- If the system supports hardware-level power tracing (e.g., via RAPL), `CodeCarbon` or `CarbonTracker` can be used.

The pseudocode below summarizes the main logic:

```
if model.type == 'LLM' and hardware == 'NVIDIA A100':
  use_tool = 'LLMCarbon'
elif execution == 'cloud' and vendor in ['AWS', 'GCP']:
  use_tool = 'CloudEstimator'
elif power_tracing == True:
  use_tool = 'CodeCarbon'
else:
  use_tool = 'GreenAlgorithms'
```

The procedure is designed to be easily embedded into AI pipelines and is extensible to new tools as they emerge. Table 4 provides an overview of which tools are suitable for each model category under different execution scenarios. This logic is extensible and can be integrated with declarative rule engines or configuration-driven systems. While the current version relies on hand-crafted rules, future extensions may include scoring-based multi-tool selection or probabilistic confidence estimates.

The objective of our empirical evaluation is to illustrate the heterogeneity of existing emission estimation tools and the potential of a hybrid aggregation and dynamic matching strategy to enhance consistency and interpretability. Our experiments focus on a representative variety of AI models (from tree-based to large-scale LLMs) and infrastructures (local CPU/GPU and cloud environments).

We acknowledge that this evaluation does not aim to provide statistically rigorous validation of the tools' accuracy in an absolute sense. Indeed, the absence of standardized ground-truth emission benchmarks in AI remains a known limitation in this research area. Consequently, our analysis prioritizes breadth of applicability and qualitative comparison over numerical benchmarking.

We view this work as an initial step toward a standardized and extensible methodology for the transparent evaluation of environmental impacts in AI development and deployment.

4. Experimental evaluation

In this section, a series of examples are defined in which the methods developed are compared with an established framework in the literature, such as MLCO2 as there is no universally accepted ground-truth for carbon emission estimates in AI workloads, this evaluation focuses on relative tool comparison rather than absolute validation.

In the remainder of this section, we refer to the six selected emission estimation tools using the labels E1 through E6. These identifiers correspond to the tools that were successfully applied across all experimental scenarios. Specifically:

- **E1**: CodeCarbon
- **E2**: Green Algorithms
- **E3**: ML CO_2 Impact
- **E4**: Carbontracker
- **E5**: Experiment Impact Tracker
- **E6**: OpenCarbonEval

4.1. Application and comparison of methods

(1) It is assumed that the example of GPT-3 is considered, representing a model based on the architecture of large language models, also referred to as Large Language Models (LLMs). It consists of a very large number of parameters, approximately 175 billion, and is the model also used by ChatGPT.

Regarding its characteristics, the following aspects are taken into account:

Table 5
Comparison of carbon emission estimates across different models and methods.

Model	Weighted average ($kgCO_2e$)	MLCO2 ($kgCO_2e$)	Dynamic method (Framework)
GPT-3 (LLM)	17.5	17.1	17.15 (LLMCarbon)
DenseNet-201 (DL)	15.7	16.2	15.63 (Green Algorithms)
MobileNet (ML)	11.8	22.5	6.85 (CodeCarbon)

- An NVIDIA V100 GPU is considered, as it provides sufficient computational power for a language model of this scale, featuring 16 GB of memory and a TDP of 300 W. An Intel Core i9-13900K CPU is considered with a TDP of 125 W. A 64 GB RAM is considered to support large datasets. Runtime is supposed to be 100 h. It is assumed that the same utilization rates, presented in the example of Section 3, are considered namely 80% for the GPU, 50% for the CPU, and 40% for the memory, for the frameworks that take these into account.
- The model is assumed to be deployed on an Amazon Web Services (AWS) cloud, with a geographical location in US-East. The carbon intensity for that region, which can be consulted on the Electricity Map website,[2] is assumed to be 365 gCO_2e/kWh.

(2) A **DenseNet-201** model is assumed to be trained. For this purpose, an NVIDIA A100 GPU is considered, as it provides high computational power for an advanced deep learning model, with 40 GB of memory and a TDP of 400 W. An AMD Ryzen Threadripper PRO 5995WX CPU with a TDP of 280 W is also considered. A total of 128 GB of RAM is selected to support large datasets. The estimated execution time is 150 h.

The following utilization rates are assumed: 75% for the GPU, 60% for the CPU, and 50% for memory, for frameworks that take these parameters into account.

The model is assumed to be deployed on Google Cloud Platform (GCP), with a geographic location in EU-West1, where the considered carbon intensity value is 122 CO2eq/kWh.

(3) A **MobileNet** model is considered. For this purpose, an NVIDIA Tesla K80 GPU is used, offering 24 GB of memory with a TDP of 300 W. An Intel Xeon E5-2690 v4 CPU is also considered, with a TDP of 135 W. A 64 GB RAM is used to support moderately sized datasets. The expected execution time is 250 h.

It is assumed that the following utilization rates are considered: 70% for the GPU, 50% for the CPU, and 40% for the memory, for frameworks that take these parameters into account.

The model is assumed to be deployed on Amazon Web Services (AWS) cloud, with a geographic location in US-West (Oregon). The carbon intensity for that region can be consulted on the Electricity Map website; in this case the value is 46 CO2eq/kWh.

The results of the following 3 cases are summarized in the Table 5.

5. Discussion

The results obtained from the experimental evaluation highlight the strengths and limitations of each proposed methodology for estimating carbon emissions in AI models. Three distinct approaches were compared: the weighted average framework, the MLCO2 benchmark, and the dynamic matching method. Each was applied to three representative deep learning architectures (GPT-3, DenseNet-201, and MobileNet) deployed in diverse computational environments.

The weighted average method provides a robust and balanced estimation across different models, yielding values that are consistently aligned with the average behavior of all tools considered. This confirms its effectiveness in minimizing the influence of outliers and compensating for the limitations of individual tools. In the GPT-3 case, for example, the weighted average result (17.5 $kgCO_2e$) is slightly higher than that of MLCO2 (17.1), while remaining close to the outcome of the dynamic method (17.15 through LLMCarbon), suggesting a high level of reliability.

The MLCO2 tool, while widely used and accessible, tends to underestimate or overestimate emissions depending on the specific model architecture and its hardware assumptions. For MobileNet, MLCO2 estimated the highest value (22.5 $kgCO_2e$), which appears inconsistent with the simpler architecture and lower computational load, as confirmed by both the weighted average (11.8) and the dynamic method (6.85 with CodeCarbon). This indicates that MLCO2 may not fully account for hardware efficiency or optimized runtime conditions in smaller models.

The dynamic method, based on model-feature-to-tool matching, shows significant adaptability. It tends to select the most appropriate framework based on the model's characteristics, such as parameter count, infrastructure type (cloud vs. local), and hardware details. For example, it correctly associates LLMCarbon with GPT-3, Green Algorithms with DenseNet-201 and CodeCarbon with MobileNet, resulting in emission values that are not only credible but also computationally justifiable. Furthermore, in the case of MobileNet, the dynamic method produced the lowest estimated impact, suggesting its sensitivity to the low weight architectures.

Overall, the results confirm that no single tool is universally optimal. The proposed weighted average strategy ensures stability and broad applicability, while the dynamic method excels in context-aware estimations. MLCO2, though commonly used, should be complemented with additional tools or strategies in order to improve accuracy, especially in edge cases or highly optimized configurations.

These observations reinforce the need for flexible and hybrid methodologies in carbon impact assessments of AI systems. Furthermore, we emphasize the importance of integrating multiple perspectives: statistical, contextual, and technical, to provide transparent and realistic estimates in sustainability-oriented AI development.

On the absence of a reliable baseline

One of the key challenges in carbon footprint estimation for AI systems is the lack of a universally accepted baseline or ground truth against which to validate predictions. While some tools rely on hardware-level measurements (e.g., RAPL or wattmeter-based approaches), such data is not always accessible or standardized, especially in cloud environments.

Our evaluation does not attempt to establish the absolute accuracy of each estimation tool, but instead focuses on comparing their relative outputs and exploring how aggregation and matching strategies can improve robustness and interpretability.

Future work will aim to incorporate validation against partial baselines where available, including:

- low-level energy readings on local machines,
- APIs from cloud providers that expose energy usage,
- benchmark workloads with known energy profiles under controlled conditions.

We believe that the development of standardized benchmark scenarios and reference implementations will be essential to improve comparability and accuracy assessments in this field.

[2] https://app.electricitymaps.com/zone/US-MIDA-PJM/24h.

6. Conclusion and future work

This paper presented a dual-strategy methodology for estimating the carbon footprint of artificial intelligence systems, with the aim of improving accuracy and contextual adaptability. While numerous tools have been proposed in recent years to quantify emissions from AI workloads they often leads to inconsistent or incomparable results. To address this, we introduced a hybrid approach combining a static, weighted aggregation of ten prominent emission trackers with a dynamic framework capable of matching AI models to the most appropriate estimation tool based on operational and architectural criteria.

The weighted aggregation method leverages six evaluation dimensions — authority, precision, completeness, real-time adaptability, implementation ease, and accessibility — to assign a score to each tracking tool. This scoring system enables the calculation of a normalized average emission estimate that compensates for individual tool limitations and biases.

Complementing this, the dynamic model-to-tool matching framework provides a mechanism to associate specific AI models with the emission tracker best suited to their context.

Our evaluation indicates that no single tool consistently performs best across all use cases. Instead, meaningful carbon estimation benefits from methodological diversity, empirical grounding, and context-sensitive adaptation.

We acknowledge the methodological limitations of our empirical section, as discussed in Section 3, and propose this framework as a transparent, extensible basis for future quantitative refinement.

On generalizability and scalability.

While the empirical evaluation presented in this paper covers a representative set of AI models and infrastructures, we acknowledge that it does not encompass the full diversity of AI workloads and environments.

Our claims of generalizability and scalability refer to the architecture of the proposed framework itself. Specifically, the scoring and weighting system, the dynamic tool-matching logic, and the aggregation strategy are designed to be modular and extensible.

New estimation tools can be added by updating the scoring matrix, and new model categories can be integrated by expanding the rule-based matching system. This enables the framework to scale as the ecosystem of carbon estimation methods and AI architectures evolves.

Future work will extend the empirical validation to additional domains such as reinforcement learning, distributed training setups, and deployment on edge devices, which pose unique challenges for carbon footprint estimation.

Future research will be directed toward three main directions. First, we plan to formalize the dynamic assignment procedure using rule-based and learning-based classifiers trained on metadata derived from model configurations and training logs. This would allow the automation of tool selection based on input parameters. Second, we aim to extend the methodology to account for emissions over the entire AI model lifecycle, including data preprocessing and post-deployment inference, which are often neglected despite their cumulative impact. Third, we envision the development of a modular software library that integrates both the weighted and dynamic approaches, enabling reproducibility, configurability, and interoperability with standard machine learning workflows.

One limitation of our approach lies in the aggregation of heterogeneous estimators. Despite our weighting system accounting for tool-specific scope and adaptability, the lack of standardized measurement assumptions across tools introduces unavoidable variance. Future research should investigate harmonization techniques to improve comparability and calibrate results against common baselines.

In closing, we argue that sustainability in AI should not rely solely on improvements in hardware efficiency or green energy usage. Rather, it must also be supported by methodologies for quantifying environmental impact. This work contributes in that direction, offering a structured yet flexible approach to emission estimation that can evolve alongside emerging AI paradigms and infrastructures.

CRediT authorship contribution statement

Salvatore Borraccia: Writing – original draft, Investigation, Formal analysis, Data curation. **Elio Masciari:** Writing – review & editing, Supervision, Project administration, Methodology. **Enea Vincenzo Napolitano:** Writing – review & editing, Writing – original draft, Project administration, Investigation, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used DeepL in order to have a grammar check. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] Schwartz R, Dodge J, Smith NA, Etzioni O. Green ai. Commun. ACM 2020;63:54–63.
[2] Verdecchia R, Sallou J, Cruz L. A systematic review of green ai wiley interdisciplinary reviews:. Data Min Knowl Discov 2023;13:e1507.
[3] Masciari E, Napolitano EV. Environmental sustainability of ai: Estimating co 2 e emissions across cloud, edge, and fog paradigms. In: International conference on web information systems engineering. Springer; 2024, p. 409–18.
[4] Wu C-J, Raghavendra R, Gupta U, Acun B, Ardalani N, Maeng K, Chang G, Aga F, Huang J, Bai C, et al. Sustainable ai: Environmental implications, challenges and opportunities. Proc Mach Learn Syst 2022;4:795–813.
[5] Dong M, Wang G, Han X. Impacts of artificial intelligence on carbon emissions in china: in terms of artificial intelligence type and regional differences. Sustain Cities Soc 2024;113:105682.
[6] Bolón-Canedo V, Morán-Fernández L, Cancela B, Alonso-Betanzos A. A review of green artificial intelligence: Towards a more sustainable future. Neurocomputing 2024;128096.
[7] Kar AK, Choudhary SK, Singh VK. How can artificial intelligence impact sustainability: A systematic literature review. J Clean Prod 2022;376:134120.
[8] Yokoyama AM, Ferro M, de Paula FB, Vieira VG, Schulze B. Investigating hardware and software aspects in the energy consumption of machine learning: A green ai-centric analysis. Concurr Comput: Pr Exp 2023;35:e7825.
[9] Hudaszek K, Chomiak-Orsa I, AL-Dobai SAM. Reduction of carbon dioxide emissions of it hardware. In: IFIP international workshop on artificial intelligence for knowledge management. Springer; 2023, p. 237–51.
[10] Naidu R, Diddee H, Mulay A, Vardhan A, Ramesh K, Zamzam A. Towards quantifying the carbon emissions of differentially private machine learning. 2021, arXiv preprint arXiv:2107.06946.
[11] Bouza L, Bugeau A, Lannelongue L. How to estimate carbon footprint when training deep learning models? a guide and review. Environ Res Commun 2023;5:115014.
[12] García-Martín CF, Riley G, Grahn H. Estimation of energy consumption in machine learning. J Parallel Distrib Comput 2019;134:75–88.
[13] Delanoë P, Tchuente D, Colin G. Method and evaluations of the effective gain of artificial intelligence models for reducing co2 emissions. J Environ Manag 2023;331:117261.
[14] Faiz A, Kaneda S, Wang R, Osi R, Sharma P, Chen F, Jiang L. Llmcarbon: Modeling the end-to-end carbon footprint of large language models. 2023, arXiv preprint arXiv:2309.14393.

[15] Yu Z, Wu Y, Deng Z, Tang Y, Zhang X-P. Opencarboneval: A unified carbon emission estimation framework in large-scale ai models. 2024, arXiv preprint arXiv:2405.12843.
[16] Anselmo M, Vitali M. A data-centric approach for reducing carbon emissions in deep learning. In: International conference on advanced information systems engineering. Springer; 2023, p. 123–38.
[17] Salehi S, Schmeink A. Data-centric green artificial intelligence: A survey. IEEE Trans Artif Intell 2023;5:1973–89.
[18] Omar R, Bogner J, Muccini H, Lago P, Martínez-Fernández S, Franch X. The more the merrier? navigating accuracy vs. energy efficiency design trade-offs in ensemble learning systems. 2024, arXiv preprint arXiv:2407.02914.
[19] Vinuesa R, Azizpour H, Leite I, Balaam M, Dignum V, Domisch S, Felländer A, Langhans SD, Tegmark M, Nerini FFuso. The role of artificial intelligence in achieving the sustainable development goals. Nat Commun 2020;11:233.
[20] Greif L, Kimmig A, El Bobbou S, Jurisch P, Ovtcharova J. Strategic view on the current role of ai in advancing environmental sustainability: a swot analysis. Discov Artif Intell 2024;4:45.
[21] Ahmad T, Zhang D, Huang C, Zhang H, Dai N, Song Y, Chen H. Artificial intelligence in sustainable energy industry: Status quo, challenges and opportunities. J Clean Prod 2021;289:125834.
[22] Lottick K, Susai S, Friedler SA, Wilson JP. Energy usage reports: Environmental awareness as part of algorithmic accountability. 2019, arXiv preprint arXiv:1911.08354.
[23] Lacoste A, Luccioni A, Schmidt V, Dandres T. Quantifying the carbon emissions of machine learning. 2019, arXiv preprint arXiv:1910.09700.
[24] Henderson P, Hu J, Romoff J, Brunskill E, Jurafsky D, Pineau J. Towards the systematic reporting of the energy and carbon footprints of machine learning. J Mach Learn Res 2020;21:1–43.
[25] Anthony LFW, Kanding B, Selvan R. Carbontracker: Tracking and predicting the carbon footprint of training deep learning models. 2020, arXiv preprint arXiv:2007.03051.
[26] Trébaol T. Cumulator—a tool to quantify and report the carbon footprint of machine learning computations and communication in academia and healthcare. 2020.
[27] Budennyy SA, Lazarev VD, Zakharenko NN, Korovin AN, Plosskaya O, Dimitrov DV, Akhripkin V, Pavlov I, Oseledets IV, Barsola IS, et al. Eco2ai: carbon emissions tracking of machine learning models as the first step towards sustainable ai. In: Doklady mathematics. Vol. 106, Springer; 2022, p. S118–28.
[28] Lannelongue L, Grealey J, Inouye M. Green algorithms: quantifying the carbon footprint of computation. Adv Sci 2021;8:2100707.
[29] Dodge J, Prewitt T, Combes RTachetdes, Odmark E, Schwartz R, Strubell E, Luccioni AS, Smith NA, DeCario N, Buchanan W. Measuring the carbon intensity of ai in cloud instances. In: Proceedings of the 2022 ACM conference on fairness, accountability, and transparency. 2022, p. 1877–94.
[30] Moro A, Lonza L. Electricity carbon intensity in european member states: Impacts on ghg emissions of electric vehicles. Transp Res Part D: Transp Environ 2018;64:5–14.